

IF3130

Sistem Paralel dan Terdistribusi

parallel programming model: Hybrid – MPI + OpenMP

Achmad Imam Kistijantoro (imam@staff.stei.itb.ac.id)

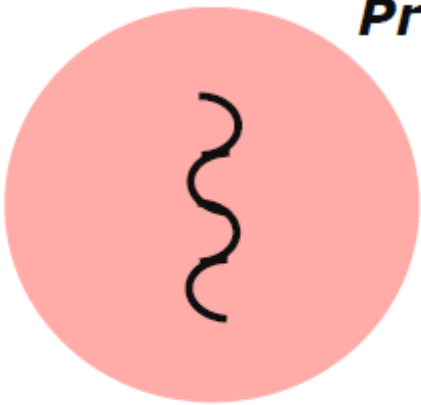
Robithoh Annur (robithoh@staff.stei.itb.ac.id)

Andreas Bara Timur (bara@staff.stei.itb.ac.id)

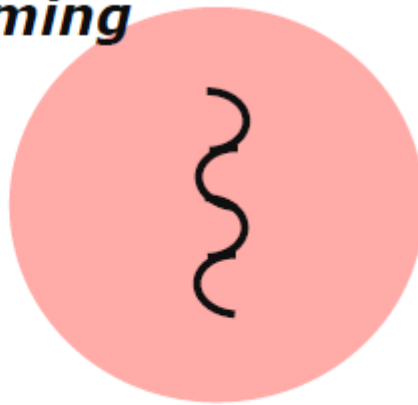
Hybrid MPI dan OpenMP

- ▶ MPI dapat digunakan bersamaan dengan OpenMP
 - ▶ MPI untuk parallel antar node
 - ▶ OpenMP untuk parallel dengan multithreading

MPI-only Programming



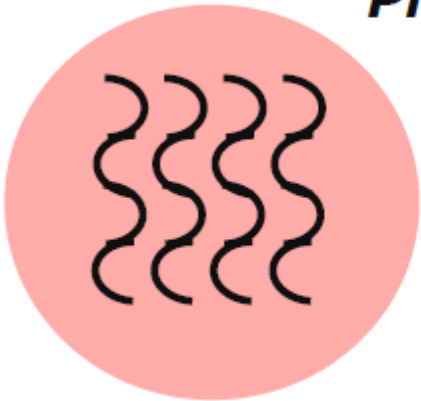
***Rank
0***



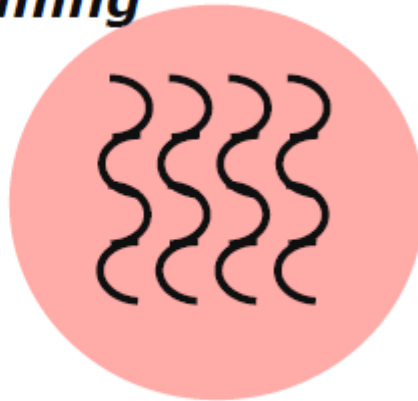
***Rank
1***

-
- ▶ Pada hybrid MPI+OpenMP, beberapa threads dapat berjalan Bersama
 - ▶ Share MPI objects (communicators, requests)

MPI+Threads Hybrid Programming



***Rank
0***



***Rank
1***

MPI Thread Safety

- ▶ Ada 4 level thread safety yang disediakan MPI
 - ▶ `MPI_THREAD_SINGLE`: hanya 1 thread
 - ▶ `MPI_THREAD_FUNNELED`: multithreaded, tapi hanya 1 thread yang memanggil MPI
 - ▶ `MPI_THREAD_SERIALIZED`: multithreaded, tapi hanya 1 thread pada setiap saat yang memanggil MPI
 - ▶ `MPI_THREAD_MULTIPLE`: multithreaded, dan setiap thread dapat memanggil MPI
- ▶ Aplikasi yang dapat berjalan menggunakan sebuah level, dapat berjalan menggunakan level yang lebih tinggi (e.g. jika aman menggunakan `SINGLE`, akan aman menggunakan `FUNNELED`)
- ▶ konfigurasi: `MPI_Init_thread(requested, provided)`

MPI_THREAD_SINGLE

- ▶ Hanya 1 thread yang berjalan

```
int main(int argc, char ** argv)
{
    int buf[100];
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    for (i = 0; i < 100; i++)
        compute(buf[i]);
    /* Do MPI stuff */
    MPI_Finalize();
    return 0;
}
```

MPI_THREAD_FUNNELED

- ▶ Semua pemanggilan MPI oleh master thread

```
int main(int argc, char ** argv)
{
    int buf[100], provided;
    MPI_Init_thread(&argc, &argv, MPI_THREAD_FUNNELED, &provided);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    #pragma omp parallel for
    for (i = 0; i < 100; i++)
        compute(buf[i]);

    /* Do MPI stuff */
    MPI_Finalize();
    return 0;
}
```

MPI_THREAD_SERIALIZED

- ▶ Hanya 1 thread yang memanggil MPI

```
int main(int argc, char ** argv)
{
    int buf[100], provided;
    MPI_Init_thread(&argc, &argv, MPI_THREAD_SERIALIZED, &provided);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    #pragma omp parallel for
    for (i = 0; i < 100; i++) {
        compute(buf[i]);
        #pragma omp critical
        /* Do MPI stuff */
    }
    MPI_Finalize();
    return 0;
}
```

MPI_THREAD_MULTIPLE

- ▶ Setiap thread dapat memanggil MPI

```
int main(int argc, char ** argv)
{
    int buf[100], provided;
    MPI_Init_thread(&argc, &argv, MPI_THREAD_MULTIPLE, &provided);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    #pragma omp parallel for
    for (i = 0; i < 100; i++) {
        compute(buf[i]);
        /* Do MPI stuff */
    }
    MPI_Finalize();
    return 0;
}
```

Implementasi

- ▶ Tidak semua implementasi MPI menyediakan support di atas `MPI_THREAD_SINGLE`
- ▶ Implementasi yang thread-safe menyediakan support `MPI_THREAD_MULTIPLE`
- ▶ Default (jika menggunakan `MPI_Init`):
`MPI_THREAD_SINGLE`

Spesifikasi MPI_THREAD_MULTIPLE

- ▶ **Ordering:** saat beberapa thread memanggil MPI secara konkuren, hasilnya akan sama seperti jika pemanggilan MPI tersebut dilakukan serial dengan order tertentu
 - ▶ Order dijamin dalam 1 thread yang sama
 - ▶ User harus menjamin bahwa pemanggilan kolektif dalam 1 komunikator berjalan dengan urutan yang sama
 - ▶ User harus menjamin pemanggilan MPI yang dilakukan antar thread tidak mengakibatkan race condition
- ▶ **Blocking:** pemanggilan MPI yang blocking hanya akan memblock thread yang memanggil